POO-PIX

Rapport de projet de fin d'année

GameGirl (Emulateur GameBoy)

Samy SMAIL
Fouâd-Eddine AIT-MEDDOUR

Supervisé Par :

Prof. Tarek MILITTI

April 24, 2026

Departement Informatique

Université Évry Paris-Saclay

Chapter 1

Remerciement

Nous tenons à remercier M. Tarek Melliti pour nous avoir confié ce projet et pour son encadrement tout au long du semestre.
Nous souhaitons également remercier chaleureusement la communauté gbdev et les contributeurs de Pan Docs[1], sans qui ce projet n'aurait tout simplement pas existé. Pan Docs est la documentation technique de référence de la Game Boy, et nous l'avons consultée tellement de fois que nous aurions pu la réciter de mémoire. En un mot, Pan Docs a été le troisième membre de notre groupe.

[1] https://gbdev.io/pandocs/

Contents

1 Remerciement **I**

List of Figures **IV**

2 Introduction **1**

3 La Game Boy et son histoire **2**
3.1 Histoire . . . 2
3.2 Architecture matérielle . . . 2
3.3 Les différentes versions . . . 3

4 L'Étude de l'existant **4**
4.1 Émulateurs existants . . . 4
4.1.1 mGBA . . . 4
4.1.2 SameBoy . . . 4
4.2 Ce qui manque . . . 4
4.2.1 Absence d'implémentation en Java . . . 4
4.2.2 Absence d'architecture MVC . . . 5

5 L'Étude du Besoin **6**
5.1 Besoins fonctionnels . . . 6
5.1.1 Émulation du processeur . . . 6
5.1.2 Émulation graphique . . . 6
5.1.3 Émulation audio . . . 6
5.1.4 Gestion de la mémoire . . . 6
5.1.5 Gestion des entrées . . . 6
5.1.6 Chargement de ROMs . . . 7
5.2 Besoins non-fonctionnels . . . 7
5.2.1 Performance . . . 7
5.2.2 Précision d'émulation . . . 7

6 Les technologies utilisées **8**
6.1 Java 21 . . . 8
6.2 Swing . . . 8
6.3 Gradle . . . 8
6.4 Lombok . . . 8

7 L'Architecture **9**
7.1 Diagramme de classes . . . 9
7.2 Diagramme de séquence . . . 10

8 L'implémentation **11**
8.1 CPU . . . 11
8.1.1 Fonctionnement . . . 11
8.1.2 Notre implémentation . . . 11
8.2 PPU . . . 12

8.2.1 Fonctionnement 12
8.2.2 Notre implémentation 12
8.3 APU 12
8.3.1 Fonctionnement 12
8.3.2 Notre implémentation 12
8.4 Memory Bank Controller 12
8.4.1 Fonctionnement 12
8.4.2 Notre implémentation 13
8.5 Timer 13
8.5.1 Fonctionnement 13
8.5.2 Notre implémentation 13
8.6 Joypad 13
8.6.1 Fonctionnement 13
8.6.2 Notre implémentation 13

9 Le Résultat finale **14**
9.1 Sélection de la ROM 14
9.2 Interface principale 15
9.3 Vue cartouche 17
9.4 Vue des tuiles 18
9.5 Logo de l'exécutable 18

10 Conclusion **19**

List of Figures

Figure 7.1 Diagramme de classes 9
Figure 7.2 Diagramme de séquence 10

Figure 9.1 Navigateur de fichiers 14
Figure 9.2 Écran d'accueil de Tetris 15
Figure 9.3 Menu de sélection du type de jeu 15
Figure 9.4 Partie en cours 16
Figure 9.5 Partie en cours avec les valeurs des registres mises à jour 16
Figure 9.6 Écran de pause 17
Figure 9.7 Informations de la cartouche Tetris 17
Figure 9.8 Vue des tuiles chargées en VRAM 18
Figure 9.9 Logo de Gamegirl Emu 18

Chapter 2

Introduction

Dans le cadre de l'unité d'enseignement *Programmation Orientée Objet* et *PIX*, enseignée en deuxième année de licence Informatique à l'Université d'Évry Paris-Saclay, un projet nous a été confié par *M. Tarek Melliti*[1], portant sur la conception et le développement d'un émulateur de la console Game Boy originale (DMG-01).

Notre objectif principal est de reproduire fidèlement le comportement matériel de la Game Boy, en implémentant ses différents composants : le processeur (CPU Sharp LR35902), l'unité de traitement graphique (PPU), l'unité de traitement audio (APU), la gestion mémoire ainsi que les périphériques d'entrée. Le tout en Java, en appliquant les principes de la programmation orientée objet.

La présentation du projet s'articulera autour de trois grandes parties :

- L'étude de l'existant et l'analyse des besoins du projet.
- La présentation des technologies utilisées et les choix d'architecture logicielle.
- L'implémentation détaillée des composants matériels émulés, accompagnée des diagrammes de classes et de séquence.

[1] Tarek Melliti : https://www.ibisc.univ-evry.fr/ tmelliti/

Chapter 3

La Game Boy et son histoire

3.1 Histoire

La Game Boy est une console portable de jeu vidéo 8 bits développée et fabriquée par Nintendo, mise en vente au Japon le 21 avril 1989, puis en Amérique du Nord en juillet 1989, et enfin en Europe en 1990. Elle fut conçue par Gunpei Yokoi et l'équipe Nintendo Research & Development [1].

Malgré la sortie de consoles portables techniquement plus avancées, la Game Boy connaît un franc succès. Les modèles Game Boy et Game Boy Color totalisent 118,6 millions d'exemplaires vendus à travers le monde. Dès sa sortie aux États-Unis, plus d'un million d'exemplaires sont vendus en quelques semaines. Son succès repose sur trois facteurs principaux : une autonomie exceptionnelle, un prix accessible et un catalogue de jeux riche, dont le célèbre Tetris inclus dans chaque bundle.

3.2 Architecture matérielle

La Game Boy originale, aussi appelée DMG (Dot Matrix Game), est dotée des caractéristiques techniques suivantes :

- **Processeur** : Sharp LR35902, processeur 8 bits cadencé à 4,194304 MHz
- **RAM** : 8 Ko de mémoire de travail (WRAM)
- **VRAM** : 8 Ko de mémoire vidéo
- **Écran** : LCD 2,6 pouces, 160x144 pixels en 4 niveaux de gris
- **Audio** : stéréo, 2 canaux pulse, 1 canal wave 4 bits programmable, 1 canal noise

[1] Source : https://fr.wikipedia.org/wiki/Game_Boy

3.3 Les différentes versions

Au fil des années, Nintendo a commercialisé plusieurs versions de la Game Boy, dont les plus connues sont la Game Boy Color (GBC) en 1998 et la Game Boy Advance (GBA) en 2001. Notre projet se concentre uniquement sur la Game Boy DMG originale de 1989.

Chapter 4

L'Étude de l'existant

4.1 Émulateurs existants

4.1.1 mGBA

mGBA[1] est un émulateur open source développé en C, supportant la Game Boy, la Game Boy Color ainsi que la Game Boy Advance. Il est reconnu pour sa grande précision d'émulation et sa compatibilité avec la quasi-totalité des ROMs commerciales. Il est activement maintenu et disponible sur toutes les plateformes majeures.

4.1.2 SameBoy

SameBoy[2] est un émulateur open source développé en C, axé sur la précision d'émulation de la Game Boy et de la Game Boy Color. Il est considéré comme l'un des émulateurs les plus précis disponibles à ce jour, et est souvent utilisé comme référence par les développeurs souhaitant créer leurs propres jeux sur Game Boy.

4.2 Ce qui manque

4.2.1 Absence d'implémentation en Java

La quasi-totalité des émulateurs Game Boy existants sont développés en C ou C++, des langages bas niveau offrant de hautes performances. Il n'existe pratiquement aucun émulateur Game Boy développé en Java, langage orienté objet de haut niveau enseigné dans le cadre de nos modules POO et PIX. Notre projet comble ce manque en proposant une implémentation complète en Java.

[1] mGBA: https://mgba.io/
[2] SameBoy: https://sameboy.github.io/

4.2.2 Absence d'architecture MVC

Les émulateurs existants sont généralement conçus avec une priorité donnée aux performances, au détriment de la lisibilité et de l'organisation du code. Notre projet adopte une architecture Modèle-Vue-Contrôleur (MVC), séparant clairement la logique d'émulation (Model), l'interface graphique (Vue) et les interactions utilisateur (Controller), ce qui en fait un projet plus maintenable et plus adapté à un contexte pédagogique.

Chapter 5

L'Étude du Besoin

5.1 Besoins fonctionnels

5.1.1 Émulation du processeur

L'émulateur doit reproduire fidèlement le comportement du processeur Sharp LR35902 de la Game Boy, en implémentant l'intégralité de son jeu d'instructions, la gestion des interruptions et le mode HALT.

5.1.2 Émulation graphique

L'émulateur doit reproduire le comportement de l'unité de traitement graphique (PPU) de la Game Boy, incluant le rendu des tuiles de fond, des sprites, de la fenêtre, ainsi que la gestion des modes OAM, XFER, HBlank et VBlank.

5.1.3 Émulation audio

L'émulateur doit reproduire le comportement de l'unité de traitement audio (APU) de la Game Boy, incluant les quatre canaux sonores : les deux canaux pulse, le canal wave et le canal noise, ainsi que le mixage stéréo.

5.1.4 Gestion de la mémoire

L'émulateur doit implémenter le bus mémoire de la Game Boy, incluant la gestion des différents types de Memory Bank Controllers (MBC1, MBC2, MBC3, MBC5) permettant de supporter la majorité des cartouches commerciales.

5.1.5 Gestion des entrées

L'émulateur doit permettre à l'utilisateur d'interagir avec le jeu via le clavier, en simulant les boutons de la Game Boy originale.

5.1.6 Chargement de ROMs

L'émulateur doit permettre le chargement de fichiers ROM au format `.gb` et afficher les informations de la cartouche (titre, type, taille ROM/RAM, région).

5.2 Besoins non-fonctionnels

5.2.1 Performance

L'émulateur doit tourner à une vitesse suffisante pour émuler la Game Boy en temps réel, soit 60 images par seconde, sans dégradation notable des performances sur une machine standard.

5.2.2 Précision d'émulation

L'émulateur doit être suffisamment précis pour faire tourner les ROMs commerciales majeures de manière correcte, notamment Tetris, utilisé comme ROM de référence pour les tests.

Chapter 6

Les technologies utilisées

6.1 Java 21

Java[1] est un langage de programmation orienté objet de haut niveau, développé par Oracle. Notre projet utilise Java 21, la dernière version LTS (Long Term Support) du langage, qui offre de nombreuses améliorations en termes de performance et de lisibilité du code. Il constitue le langage principal de notre émulateur.

6.2 Swing

Swing[2] est une bibliothèque graphique intégrée à Java, permettant de créer des interfaces graphiques de bureau. Elle est utilisée dans notre projet pour afficher la fenêtre de jeu, la vue des tuiles, les registres du processeur ainsi que les informations de la cartouche.

6.3 Gradle

Gradle[3] est un outil de build automation open source. Il est utilisé dans notre projet pour gérer les dépendances, compiler le code source et exécuter les tests. Notre projet utilise Gradle avec le DSL Kotlin pour la configuration du build.

6.4 Lombok

Lombok[4] est une bibliothèque Java qui permet de réduire le code répétitif en générant automatiquement les getters, setters et autres méthodes courantes via des annotations. Elle est utilisée dans notre projet pour alléger le code des classes du modèle.

[1] Java: https://www.java.com/
[2] Swing: https://docs.oracle.com/javase/tutorial/uiswing/
[3] Gradle: https://gradle.org/
[4] Lombok: https://projectlombok.org/

Chapter 7

L'Architecture

7.1 Diagramme de classes

Le diagramme de classes ci-dessous présente l'architecture générale de l'émulateur, organisée selon le patron de conception MVC.

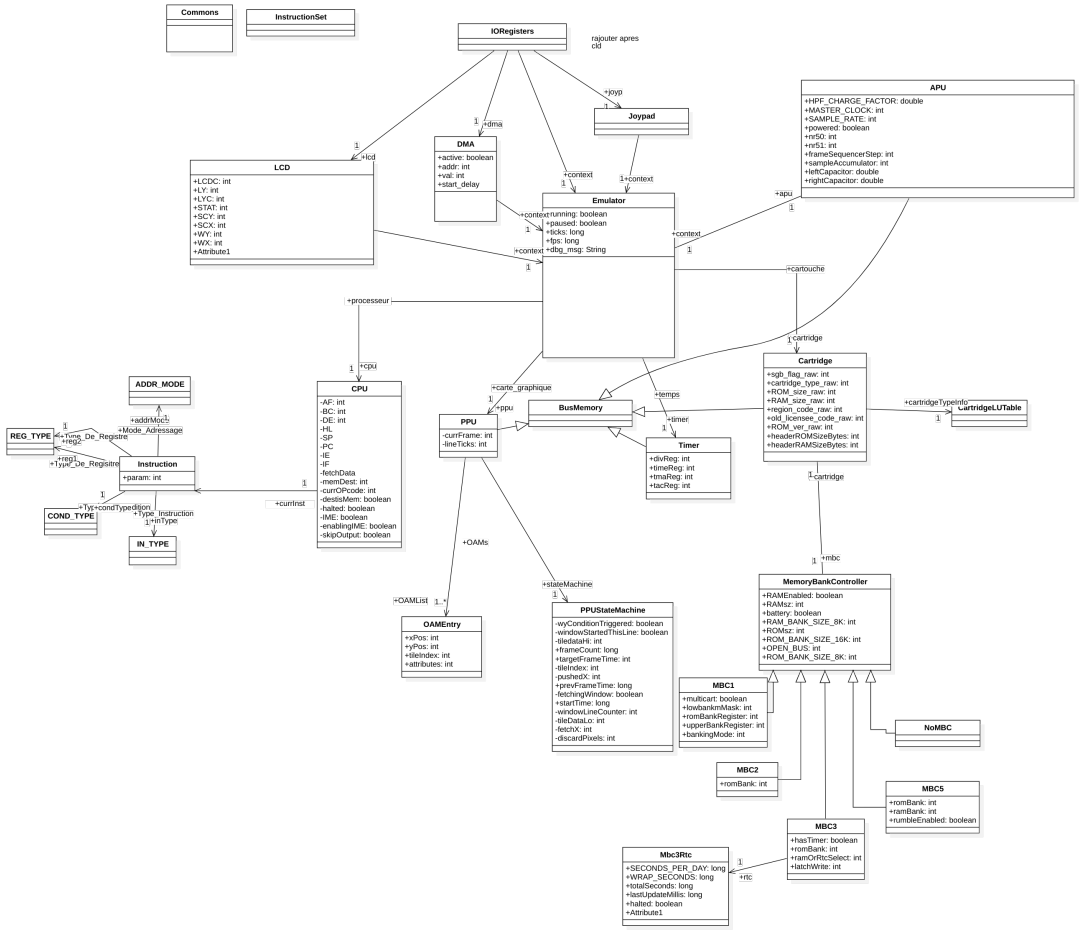

Figure 7.1: Diagramme de classes

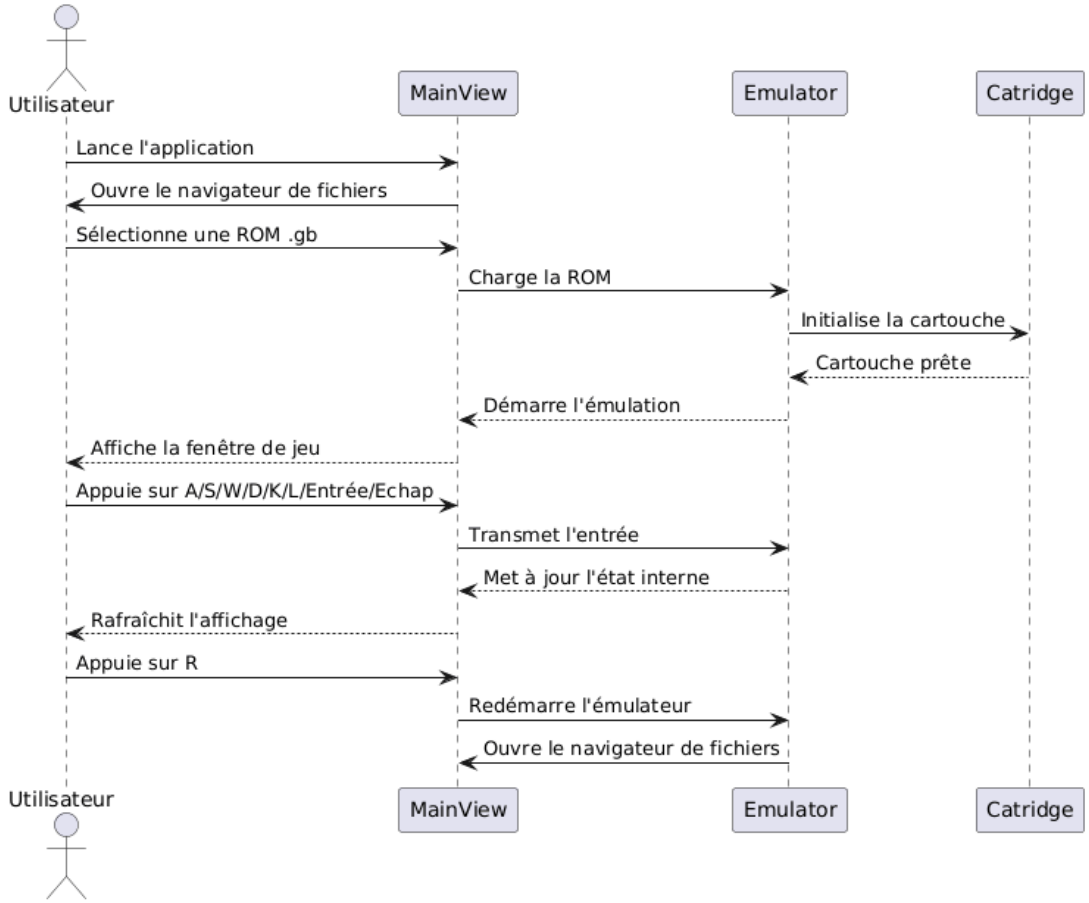

Figure 7.2: Diagramme de séquence

7.2 Diagramme de séquence

Le diagramme de séquence suivant illustre le démarrage de l'émulateur, depuis le lancement de l'application jusqu'au début d'une session de jeu.

Chapter 8

L'implémentation

Dans cette section, nous allons décrire le fonctionnement de chaque composant de la Game Boy DMG tel que documenté dans Pan Docs [1], la référence technique principale que nous avons utilisée tout au long du développement, et expliquer comment nous l'avons implémenté dans notre projet.

8.1 CPU

8.1.1 Fonctionnement

Le processeur de la Game Boy, le SM83, possède un jeu d'instructions CISC à longueur variable. Le premier octet de chaque instruction est appelé l'opcode. Le processeur dispose de plusieurs registres : AF (accumulateur et flags), BC, DE, HL, SP (pointeur de pile) et PC (compteur de programme). La plupart de ces registres peuvent être accédés soit comme un registre 16 bits, soit comme deux registres 8 bits séparés. Le registre F contient quatre flags : Zero (Z), Subtract (N), Half Carry (H) et Carry (C), qui sont mis à jour après chaque instruction et utilisés par les sauts conditionnels.

8.1.2 Notre implémentation

Nous avons implémenté le CPU dans la classe `CPU`, en suivant le cycle classique Fetch-Decode-Execute via la méthode `step()`. L'ensemble des instructions est défini dans `InstructionSet` et `IN_TYPE`, avec 21 modes d'adressage gérés par l'enum `ADDR_MODE`. Les interruptions (VBlank, LCD STAT, Timer, Serial, Joypad) sont gérées via les registres `IE` et `IF`, et le mode HALT suspend l'exécution jusqu'à réception d'une interruption.

[1] Pan Docs : https://gbdev.io/pandocs/

8.2 PPU

8.2.1 Fonctionnement

Le PPU fonctionne selon quatre modes qui se succèdent. Un cycle complet à travers ces modes prend 456 T-cycles par ligne. Le VBlank dure 4560 T-cycles, et un rafraîchissement complet de l'écran s'effectue en 70224 T-cycles. Les quatre modes sont : le scan OAM (mode 2) où les sprites visibles sont évalués, le rendu (mode 3) où les pixels sont dessinés, le HBlank (mode 0) qui est une période de repos horizontale, et le VBlank (mode 1) qui est une période de repos verticale déclenchant une interruption.

8.2.2 Notre implémentation

Le PPU est implémenté dans les classes `PPU` et `PPUStateMachine`. Il produit un framebuffer de 160×144 pixels en 4 niveaux de gris. La machine à états gère les quatre modes via l'enum `RENDER_MODE`. Le rendu des pixels utilise un pixel FIFO qui traite les tuiles de fond, la fenêtre et les sprites en respectant leurs priorités d'affichage et leurs attributs de flip.

8.3 APU

8.3.1 Fonctionnement

La Game Boy dispose de quatre canaux audio : deux canaux à onde carrée (Pulse), un canal wave et un canal noise. Chaque canal dispose d'un contrôle de longueur et d'une enveloppe de volume. Le canal Pulse 1 dispose en plus d'un effet de sweep de fréquence. Le mixage stéréo est contrôlé par les registres NR50 et NR51.

8.3.2 Notre implémentation

L'APU est implémenté dans la classe `APU` avec trois classes dédiées : `PulseChannels`, `WaveChannel` et `NoiseChannel`. Le frame sequencer est synchronisé sur le bit 13 du registre DIV interne, conformément au comportement matériel. Un filtre passe-haut est appliqué en sortie pour éliminer les offsets DC. L'audio est produit à 44 100 Hz en 16 bits stéréo via l'API `javax.sound.sampled`.

8.4 Memory Bank Controller

8.4.1 Fonctionnement

La Game Boy dispose d'un espace d'adressage de 16 bits, ce qui limite l'accès direct à 64 Ko de mémoire. Pour permettre l'utilisation de ROMs plus grandes, les cartouches intègrent un

Memory Bank Controller qui permet de commuter entre différentes banques de ROM et de RAM.

8.4.2 Notre implémentation

Le MBC est implémenté via la classe abstraite `MemoryBankController` et ses sous-classes. La classe `Catridge` lit l'en-tête de la ROM au chargement pour déterminer automatiquement le type de MBC à instancier. Nous supportons les types NoMBC, MBC1, MBC2, MBC3 et MBC5, ce qui couvre la grande majorité des jeux Game Boy.

8.5 Timer

8.5.1 Fonctionnement

Le Timer de la Game Boy permet aux jeux de déclencher des interruptions à intervalles réguliers. Il est composé de quatre registres : DIV, qui est en réalité un compteur 16 bits interne dont seuls les 8 bits supérieurs sont exposés en mémoire, TIMA qui est le compteur principal, TMA qui est la valeur de rechargement de TIMA, et TAC qui configure la fréquence et active ou désactive le timer.

8.5.2 Notre implémentation

Le Timer est implémenté dans la classe `Timer`. La fréquence de TIMA est déterminée par TAC via une détection de front descendant sur les bits du registre DIV interne. Un débordement de TIMA génère une interruption Timer et recharge le registre avec la valeur de TMA. Le registre DIV est également utilisé par l'APU pour cadencer son frame sequencer.

8.6 Joypad

8.6.1 Fonctionnement

Les huit boutons de la Game Boy sont organisés sous forme d'une matrice 2×4. On sélectionne soit les boutons d'action, soit les touches directionnelles en écrivant dans le registre P1, puis on lit les bits 0 à 3 pour connaître l'état des boutons.

8.6.2 Notre implémentation

Le Joypad est implémenté dans la classe `Joypad`. Les boutons sont mappés sur le clavier : W, A, S, D pour le D-pad, K et L pour A et B, Entrée pour Start et Échap pour Select.

Chapter 9

Le Résultat finale

9.1 Sélection de la ROM

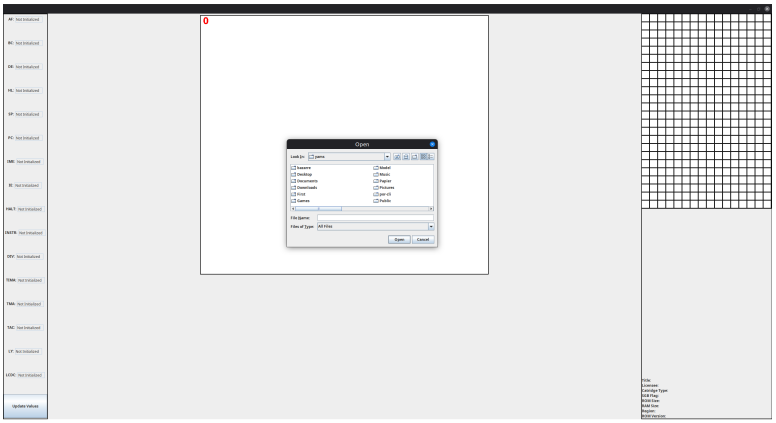

Figure 9.1: Navigateur de fichiers

9.2 Interface principale

Figure 9.2: Écran d'accueil de Tetris

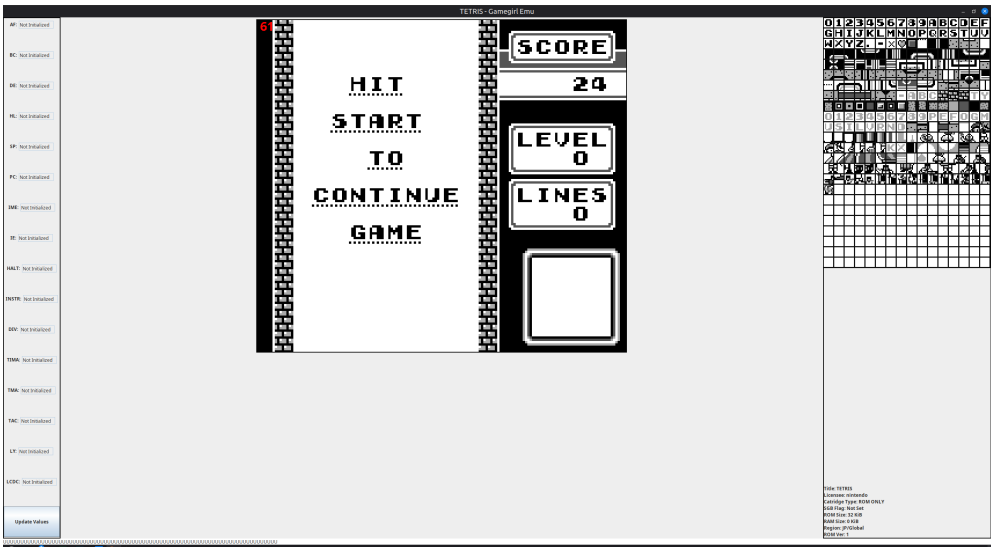

Figure 9.3: Menu de sélection du type de jeu

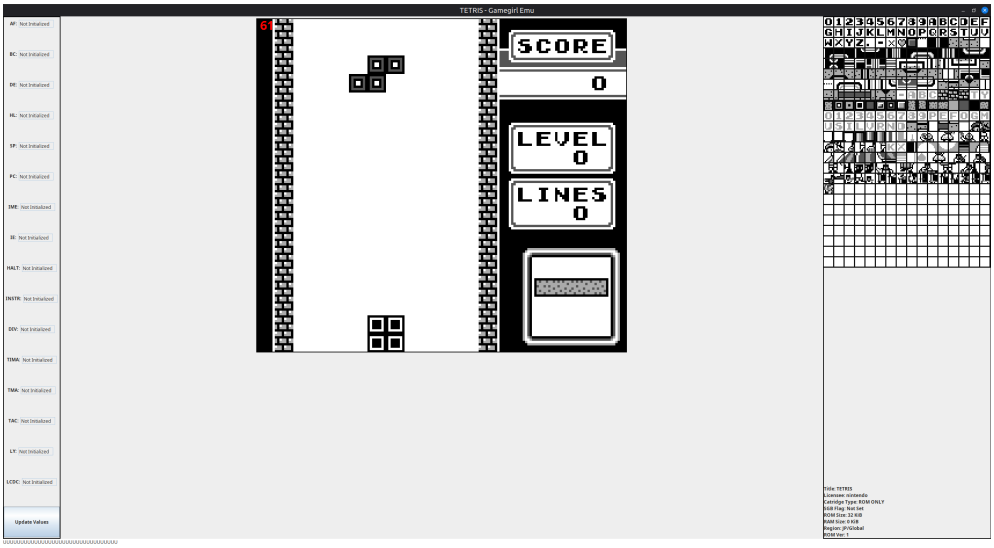

Figure 9.4: Partie en cours

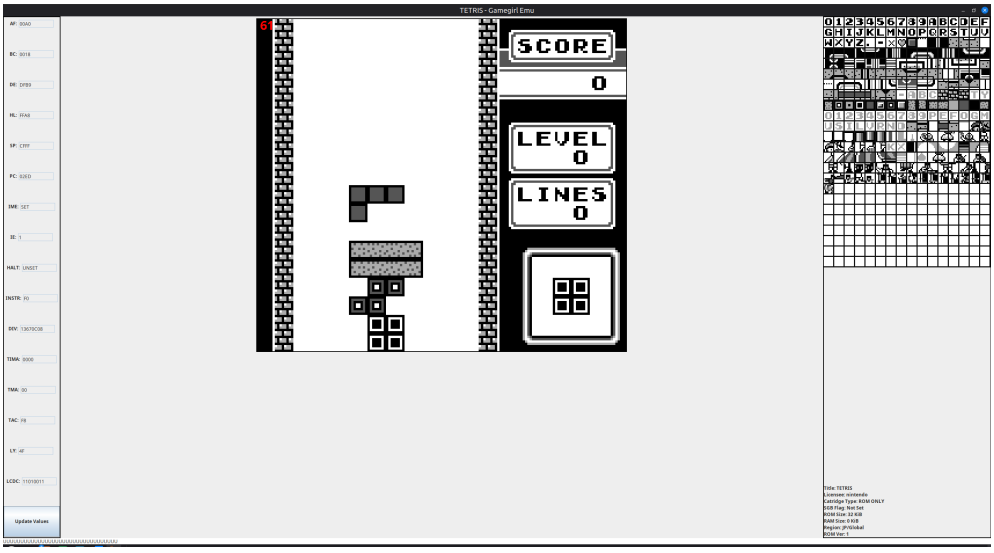

Figure 9.5: Partie en cours avec les valeurs des registres mises à jour

Figure 9.6: Écran de pause

9.3 Vue cartouche

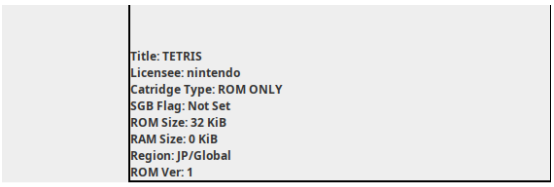

Figure 9.7: Informations de la cartouche Tetris

9.4 Vue des tuiles

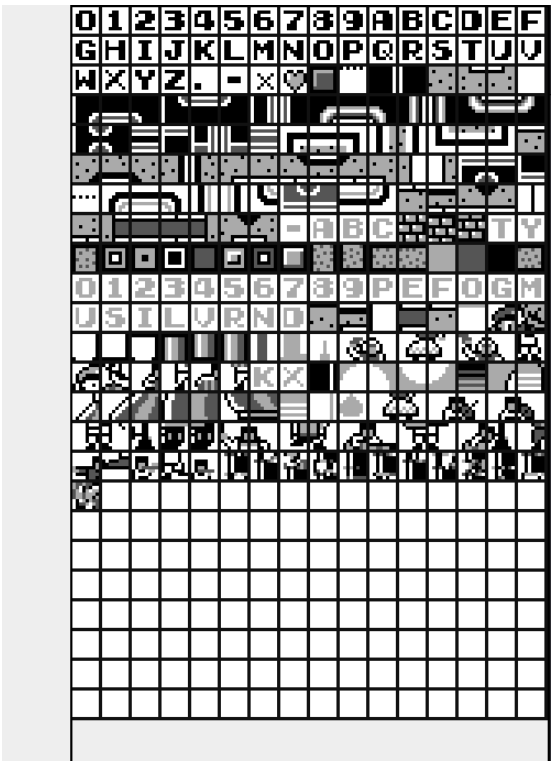

Figure 9.8: Vue des tuiles chargées en VRAM

9.5 Logo de l'exécutable

Figure 9.9: Logo de Gamegirl Emu

Chapter 10

Conclusion

Ce projet nous a permis de mettre en pratique les concepts de la programmation orientée objet enseignés dans le cadre des modules POO et PIX, en les appliquant à un domaine technique exigeant : l'émulation de matériel. Implémenter une Game Boy from scratch en Java nous a confrontés à des problématiques concrètes de bas niveau, notamment la gestion des cycles machine, la synchronisation des composants et la précision du rendu audio et vidéo.Nous sommes satisfaits du résultat obtenu, l'émulateur est fonctionnel et capable de faire tourner des jeux commerciaux comme Tetris de manière correcte. Cependant, plusieurs fonctionnalités que nous aurions aimé intégrer n'ont pas pu être réalisées faute de temps :

- Un choix de palettes de couleurs personnalisables
- Une fenêtre de débogage avancée
- Un menu de préférences utilisateur

Ces fonctionnalités constituent des perspectives d'amélioration naturelles pour une version future du projet.